Utica University | Department of Computer Science | Fall 2026

CSC 618: Information Retrieval | Week 10 Assignment

Submission 2: Part B, Controlled Experiment on IR Evaluation

November 2026

1. Results

All scores below are output from `part_b.py` (Appendix A). Precision, Recall and F1 are computed on the five documents each system returned.

Table 1: Per-query scores.

System	Query	Precision	Recall	F1	P@3	AP	RR
A	Q1	0.6000	1.0000	0.7500	0.6667	0.8056	1.0000
A	Q2	0.6000	1.0000	0.7500	0.6667	0.8056	1.0000
A	Q3	0.6000	1.0000	0.7500	0.6667	0.8056	1.0000
B	Q1	0.4000	0.6667	0.5000	0.3333	0.3333	0.5000
B	Q2	0.6000	1.0000	0.7500	0.6667	0.5889	0.5000
B	Q3	0.6000	1.0000	0.7500	0.3333	0.5333	0.5000

Table 2: Aggregate results over the three queries, plus nDCG@5 for Q1.

System	MAP	MRR	Mean P	Mean R	Mean F1	Mean P@3	nDCG@5 (Q1)
A	0.8056	1.0000	0.6000	1.0000	0.7500	0.6667	0.9305
B	0.4852	0.5000	0.5333	0.8889	0.6667	0.4444	0.5784

System A scores at least as high as System B on every metric and every query. The two systems differ mostly in ordering. On Q2 and Q3 they retrieve the same number of relevant documents, so Precision, Recall and F1 tie, and the difference shows up in AP and RR (and in P@3 for Q3). Q1 is the only query where B retrieves less, because it misses D5. A ranks a relevant document first every time, and B never does.

Table 3: Recommendation by search requirement.

Requirement	Evidence	Better
Retrieve more relevant documents	Mean Recall 1.0000 vs 0.8889; the gap comes from Q1 alone	A
Place relevant documents near the top	MAP 0.8056 vs 0.4852; MRR 1.0000 vs 0.5000	A
Handle degrees of relevance	nDCG@5 for Q1 0.9305 vs 0.5784 (one query only)	A

2. Manual verification

I worked these out by hand and compared them with the program. All match to four decimals.

Table 4: Hand calculations.

Item	Hand calculation	Program
AP, System B, Q3	Relevant at ranks 2, 4, 5: $(1/2+2/4+3/5)/3 = 1.6/3$	0.5333
MAP, System B	$(0.3333+0.5889+0.5333)/3 \approx 1.4556/3$	0.4852
F1, System B, Q1	$P = 2/5$, $R = 2/3$: $2(0.4)(0.6667)/(0.4+0.6667)$	0.5000
nDCG@5, System B, Q1	Gains 0, 3, 0, 2, 0: DCG $= 3/\log_2 3 + 2/\log_2 5 \approx 1.8928 + 0.8614$, which is 2.7541 before rounding. IDCG $= 3 + 2/\log_2 3 + 1/\log_2 4 = 4.7619$. Ratio $= 2.7541/4.7619$	0.5784

3. Analysis questions

(a) Which system has higher MAP? What does this indicate about its ranking quality?

System A, 0.8056 vs 0.4852. A places relevant documents earlier in the list. The MAP gap (0.3204) is much larger than the mean Recall gap (0.1111), so most of A's lead comes from better ordering, with only a small part from finding more documents.

(b) Which system has higher MRR? What does that tell you about the position of the first relevant result?

System A, 1.0000 vs 0.5000. A's first result is relevant for all three queries. B's first relevant result is at rank 2 every time, so a B user always sees one non-relevant document first.

(c) Do Precision and Recall always lead to the same conclusion about system performance? Explain.

No, although they agree in this experiment. They cannot disagree here, because every list has 5 documents and every query has 3 relevant ones, which makes Recall exactly $(5/3)\times$ Precision. They can split when list length varies. A system returning two documents, both relevant, has $P = 1.00$ and $R = 0.67$. One returning five documents with all three relevant among them has $P = 0.60$ and $R = 1.00$. Which is better depends on whether the task punishes wrong results or missed ones more.

(d) Why might a system with a high P@3 have a lower MAP than another system?

P@3 looks only at the top three positions and ignores their order, while MAP uses every relevant document. My results do not show the reversal, since A is never below B, but Q2 comes close: both have P@3 = 0.6667, and AP is 0.8056 for A and 0.5889 for B. A hypothetical query with six relevant documents shows the full effect. System X finds three of them at ranks 1 to 3 and no others (P@3 = 1.000, AP = 0.500). System Y finds all six at ranks 1, 3, 4, 6, 7 and 8 (P@3 = 0.667, AP = 0.758).

(e) Which system has the higher nDCG@5 for Q1? Explain how graded relevance affects the result.

System A, 0.9305 vs 0.5784. A puts the grade 3 document (D1) first, while B discounts it by placing it second and leaves out the grade 1 document (D5). Grades do not change the winner, but they do change the size of the gap. With binary gains the scores would be 0.9060 and 0.4982, a gap of 0.4078. With grades the gap is 0.3521. It narrows because B does retrieve the two most useful documents and misses only the marginal one, which graded relevance penalizes lightly.

(f) Why must both systems use the same queries and relevance judgments for a fair comparison?

So that the system is the only thing that varies. Different queries could simply be easier or harder, and different judgments would change what counts as correct. In either case a score difference could not be credited to the systems.

(g) What are the limitations of evaluating only three queries and eight documents?

One query can move a mean by a lot, and a significance test cannot show much: with three queries, even with A ahead on all of them, the smallest possible two-sided sign-flip p-value is $2/8 = 0.25$. Each system also returns 5 of the 8 documents, so a random list of five would already have an expected Recall of 0.625. Lists this short cannot show how a system behaves deep in a ranking.

(h) Why can these results not be treated as evidence of the performance of a real university search engine?

The rankings are hypothetical and did not come from a running engine, and the assignment states the judgments are for teaching and did not come from a user study. The documents and queries are about IR and machine learning, with nothing on courses, admissions or campus services. A real engine also has to deal with scale, fresh content, duplicates, misspellings and varied user intent. Speed and user satisfaction were not measured at all.

Appendix A: Code

ir_metrics.py

```
"""IR evaluation metrics shared by Part B and Part C. A ranked list is a list of doc IDs."""
import math

def precision(ranked, rel):
    return sum(d in rel for d in ranked) / len(ranked) if ranked else 0.0

def recall(ranked, rel):
    return sum(d in rel for d in ranked) / len(rel) if rel else 0.0

def f1(ranked, rel):
    p, r = precision(ranked, rel), recall(ranked, rel)
    return 2 * p * r / (p + r) if p + r else 0.0

def precision_at_k(ranked, rel, k):
    """The denominator stays k even if fewer than k documents were returned."""
    return sum(d in rel for d in ranked[:k]) / k

def reciprocal_rank(ranked, rel):
    for i, d in enumerate(ranked, 1):
        if d in rel:
            return 1 / i
    return 0.0

def average_precision(ranked, rel):
    """Relevant documents that were not retrieved contribute 0 (division by |rel|)."""
    hits, total = 0, 0.0
    for i, d in enumerate(ranked, 1):
        if d in rel:
            hits += 1
            total += hits / i
    return total / len(rel) if rel else 0.0

def dcg(gains):
    return sum(g / math.log2(i + 1) for i, g in enumerate(gains, 1))

def ndcg_at_k(ranked, grades, k):
    """grades: doc -> gain; unjudged documents have gain 0."""
    actual = dcg([grades.get(d, 0) for d in ranked[:k]])
    ideal = dcg(sorted(grades.values(), reverse=True)[:k])
    return actual / ideal if ideal else 0.0

def mean(xs):
    xs = list(xs)
    return sum(xs) / len(xs) if xs else 0.0
```

part_b.py

```
"""Part B: controlled comparison of System A and System B. Usage: part_b.py <output_dir>"""
import csv, json, sys
from ir_metrics import *

DOCS = {"D1": "Neural networks and deep learning methods for image classification",
        "D2": "BM25 ranking functions for information retrieval",
        "D3": "Convolutional neural networks for image recognition",
        "D4": "Language models and probabilistic document retrieval",
        "D5": "Image classification using machine learning",
        "D6": "BM25, query likelihood, and document ranking",
        "D7": "Unigram language models for text search",
        "D8": "Web crawling and link analysis"}
QUERIES = {"Q1": "Neural networks for image classification",
           "Q2": "BM25 information retrieval",
           "Q3": "Language models and document retrieval"}
REL = {"Q1": {"D1", "D3", "D5"}, "Q2": {"D2", "D4", "D6"}, "Q3": {"D4", "D6", "D7"}}
GRADES_Q1 = {"D1": 3, "D3": 2, "D5": 1}
RUNS = {"A": {"Q1": ["D1", "D2", "D3", "D5", "D4"], "Q2": ["D2", "D1", "D4", "D6", "D3"], "Q3": ["D4", "D2", "D6",
     "D7", "D1"]},
        "B": {"Q1": ["D2", "D1", "D4", "D3", "D6"], "Q2": ["D1", "D4", "D2", "D5", "D6"], "Q3": ["D1", "D4", "D3",
     "D7", "D6"]}}

rows = []
for s, run in RUNS.items():
    for q, r in run.items():
        rows.append(dict(System=s, Query=q, Precision=precision(r, REL[q]), Recall=recall(r, REL[q]), F1=f1(r, REL[
     q]),
                         P3=precision_at_k(r, REL[q], 3), AP=average_precision(r, REL[q]), RR=reciprocal_rank(r,
     REL[q])))
agg = {}
```

```
for s, run in RUNS.items():
    sel = [x for x in rows if x["System"] == s]
    gains = [GRADES_Q1.get(d, 0) for d in run["Q1"][:5]]
    agg[s] = dict(MAP=mean(x["AP"] for x in sel), MRR=mean(x["RR"] for x in sel),
                  mean_P=mean(x["Precision"] for x in sel), mean_R=mean(x["Recall"] for x in sel),
                  mean_F1=mean(x["F1"] for x in sel), mean_P3=mean(x["P3"] for x in sel),
                  gains_Q1=gains, DCG5_Q1=dcg(gains), IDCG5_Q1=dcg(sorted(GRADES_Q1.values(), reverse=True)),
                  nDCG5_Q1=ndcg_at_k(run["Q1"], GRADES_Q1, 5),
                  nDCG5_Q1_binary=ndcg_at_k(run["Q1"], {d: 1 for d in REL["Q1"]}, 5))

print(f"{'System':6} {'Query':5} {'Prec':>6} {'Recall':>6} {'F1':>6} {'P@3':>6} {'AP':>6} {'RR':>6}")
for x in rows:
    print(f"{x['System']:6} {x['Query']:5} " + " ".join(f"{x[k]:6.4f}" for k in ["Precision", "Recall", "F1", "P3",
      "AP", "RR"]))
for s in agg:
    print(s, {k: (round(v, 4) if isinstance(v, float) else v) for k, v in agg[s].items()})

out = sys.argv[1] if len(sys.argv) > 1 else "."
with open(f"{out}/partb_table.csv", "w", newline="") as f:
    w = csv.DictWriter(f, rows[0].keys()); w.writeheader(); w.writerows(rows)
json.dump(dict(rows=rows, agg=agg), open(f"{out}/partb_results.json", "w"), indent=1)
```